🎯 What I would define as your MVP

Your first version should answer one simple question:

> **Can a school use this system to register students, manage their basic academic information, and retrieve that information reliably?**

Don't try to build everything schools need.

MVP modules

Module	MVP?	What it should do
👨‍🎓 Students	**YES**	Register, view, edit, delete students
👩‍🏫 Teachers	**YES**	Register and manage teachers
📚 Courses/Subjects	**YES**	Create and manage subjects
🏫 Classes	**YES**	Create classes and assign students
📝 Enrollments	**YES**	Connect students to subjects/classes
📊 Basic results	**YES**	Enter and view student scores
🔐 Authentication	**YES**	Login + basic roles
📈 Dashboard	**YES**	Basic school statistics
🔎 Search/filter	**YES**	Find students/teachers quickly
🤖 AI assistant	**SMALL VERSION**	Natural-language queries
💰 Fees	**NO**	Later
📅 Attendance	**NO**	Later
🚌 Transport	**NO**	Later
📦 Inventory	**NO**	Later
📱 Parent portal	**NO**	Later
💬 SMS/WhatsApp	**NO**	Later

🧠 Advanced AI agents	**NO**	Later

Your 3-month roadmap

I'd divide the 12 weeks into **four phases**.

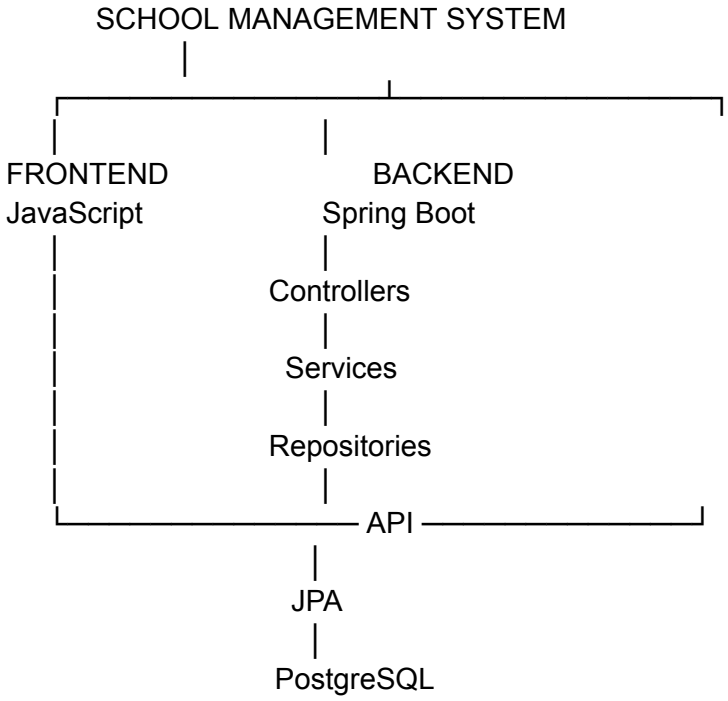
Month 1 — Build the foundation

Weeks 1–2: Backend + database

Your goal is to make Spring Boot properly communicate with PostgreSQL.

You already have a good starting point.

Your architecture should roughly become:



Learn/build:

- Entity
- DTO
- Repository
- Service
- Controller
- Validation
- Exception handling
- JPA relationships
- PostgreSQL tables
- REST APIs

Your first proper entities

Start with:

Student
Teacher
Subject
Class
Enrollment
Result
User

Don't create 30 entities.

Weeks 3–4: Student management

This becomes your **first complete vertical slice**.

For example:

Frontend
↓
POST /api/students
↓
StudentController
↓
StudentService
↓
StudentRepository
↓
PostgreSQL

A user should be able to:

Register

First name
Last name
Student number
Date of birth
Gender
Class

View

Students

ST001 Ama Mensah Form 1
ST002 Kofi Mensah Form 2
ST003 Yaa Asante Form 1

Edit

Delete

Search

View student profile

For example:

Ama Mensah

Student Number: ST001
Date of Birth: 15 April 2009
Gender: Female
Class: Form 1

Subjects

Mathematics
English
Science
ICT

Results

Mathematics 78
English 84
Science 72

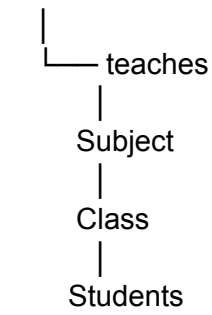
By the end of Month 1, you should have one module that genuinely works end-to-end.

That is much more valuable than having ten unfinished modules.

Month 2 — Build the school functionality

Weeks 5–6: Teachers + Subjects + Classes

Build:



For example:

Teacher
John Mensah

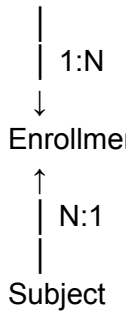
Subjects
Mathematics

Classes
Form 1A
Form 2A

Your JPA relationship learning becomes very practical here.

For example:

Student



Instead of simply learning:

> "What is @OneToMany?"

you'll understand **why your application needs it.**

Week 7: Results

Keep results extremely simple.

A teacher/admin can enter:

Student: Ama Mensah
Subject: Mathematics

CA: 25
Exam: 65

Total: 90

Then the system can display:

AMA MENSAH

Mathematics 90
English 82
Science 76
ICT 88

Average 84

Don't build a complicated grading engine yet.

Week 8: Authentication + roles

Now introduce:

User

with basic roles:

ADMIN
TEACHER

Later you can add:

STUDENT
PARENT
ACCOUNTANT
HEADMASTER

But don't do that in the MVP unless necessary.

For example:

Admin

Can:

Students
Teachers
Subjects
Classes
Results
Users

Teacher

Can:

View assigned classes
View students
Enter results

This gives your application the beginning of a real-world security model.

Month 3 — Turn it into a usable product

Month 3 should not be:

> "Let's add more features."

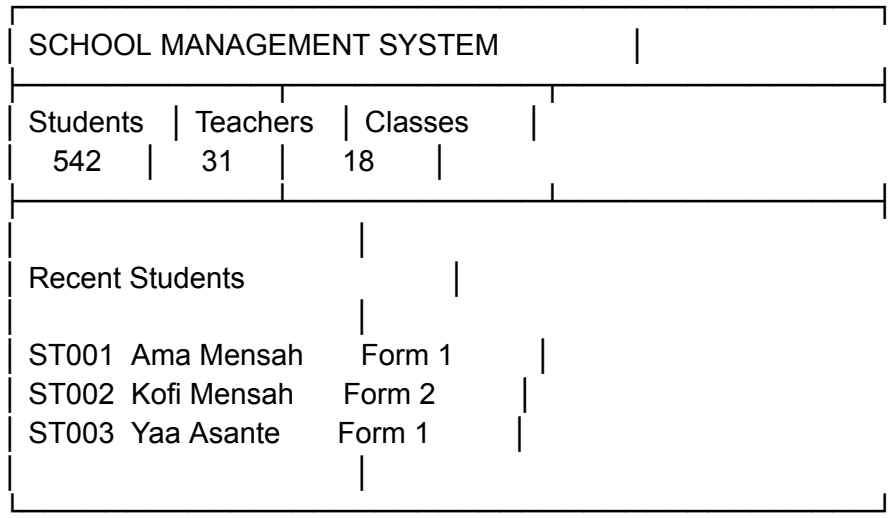
It should be:

> **"Let's make what we built actually usable."**

Week 9: Dashboard

Build a simple dashboard.

Something like:



Keep it simple.

Week 10: Search + filtering + UX

This week is extremely important.

Make the system pleasant to use.

For example:

Search students...

[Form 1 ▼] [Female ▼] [2026 ▼]

Add:

- loading states
- error messages
- success messages
- form validation
- confirmation before deletion
- empty states
- responsive design

This is where your JavaScript knowledge becomes important.

Week 11: AI experiment

This is where I would introduce your AI idea—but only as a small experiment.

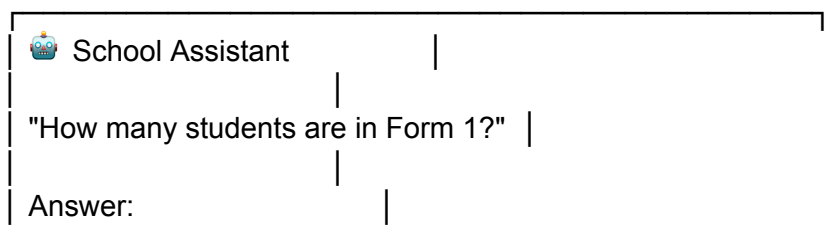
Your original long-term idea is interesting:

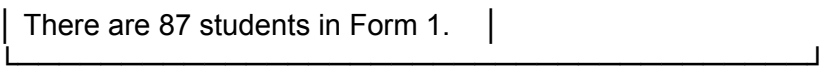
> A user talks to the school system in normal English and the AI navigates/operates the application.

Don't try to build the full agent in three months.

Instead, make a small **AI School Assistant**.

For example:



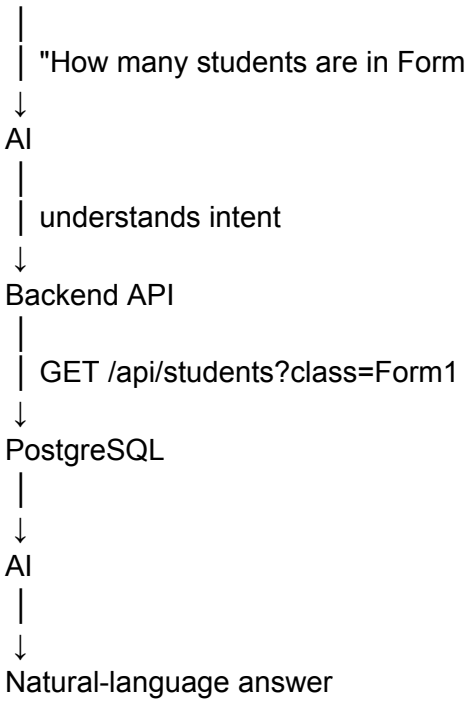


Then perhaps:

> "Show me all students in Form 2."

The AI translates the request into a controlled operation.

Conceptually:



That is enough for the MVP.

Do not let the AI directly manipulate your database.

Give it controlled APIs/tools.

Week 12: Testing + deployment + presentation

This week is about finishing.

Test:

Register student
↓
Validation
↓
Save database
↓
Display student
↓
Edit student
↓
Delete student

Test your relationships.

Test authentication.

Test API errors.

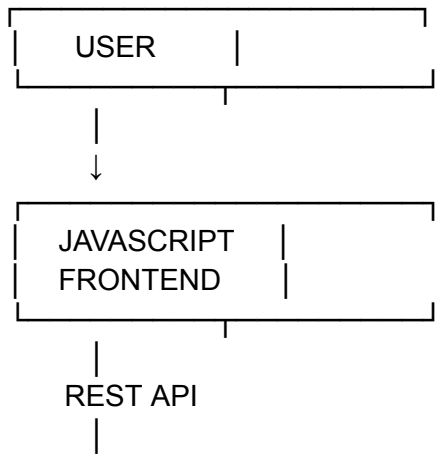
Then deploy the MVP.

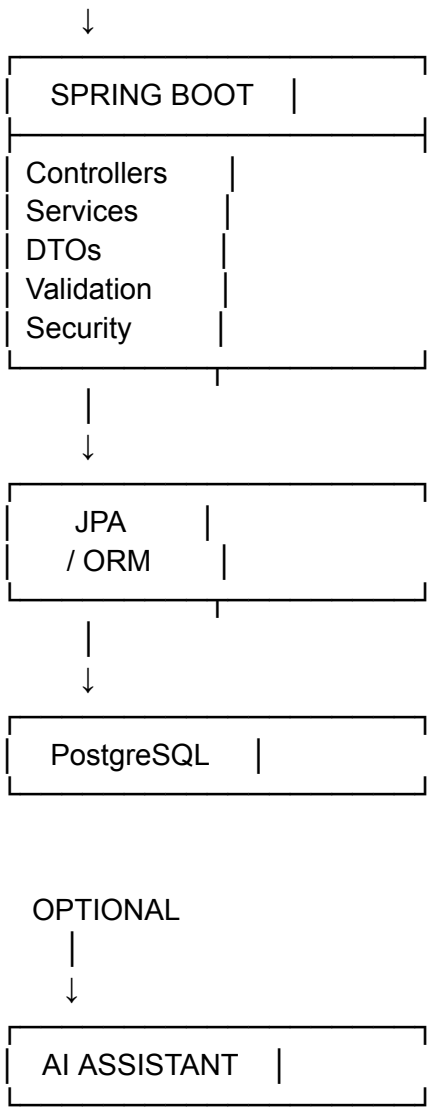
Even if it is only a small deployment, you want to be able to say:

> "Here is the School Management System. You can actually use it."

Your final MVP architecture

By the end of three months, I would like your project to look approximately like this:





Is 3 months actually reasonable?

Yes—with one important condition: scope control.

If you're trying to build:

> Student + teacher + parent + fees + attendance + timetable + library + transport + payroll + reports + mobile app + AI agent

in three months...

No.

That's not an MVP. That's the beginning of a full ERP.

But:

> Students + teachers + subjects + classes + enrollments + basic results + authentication + dashboard + small AI assistant

is a reasonable **3-month learning/product project**.

And there's another reason I think this scope fits you particularly well: you're not starting from zero. You've already been working through **JavaScript, Spring Boot, JPA, PostgreSQL and entity relationships**, so the project can serve as the place where those concepts finally connect.

Your 12-week target

I'd actually put this on your wall:

MONTH 1
═══════════════════════════════

Backend foundation
PostgreSQL
JPA
Student CRUD
REST API
Frontend ↔ Backend
═══════════════════════════════

MONTH 2
═══════════════════════════════

Teachers
Subjects
Classes
Enrollments
Results
Authentication
Roles
═══════════════════════════════

MONTH 3

Dashboard
Search/filter
UX improvements
Testing
Deployment
Small AI assistant
Documentation

And your definition of "DONE"

At the end of 12 weeks, a school administrator should be able to:

Log in → register a student → assign the student to a class → enroll them in subjects → view their profile → enter/view results → search for them → see school statistics.

And optionally ask:

> **"How many Form 1 students do we have?"**

and get an answer from your AI assistant.

That's a **real MVP**.

The important thing is that during these three months, you're not just "learning Java." You're learning **how a real software system is assembled—**frontend → API → business logic → ORM → database → authentication → AI.